Universidad Corporativa Coppel | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad | #3 | Programa Banco Mexicano (parte 2)

Lenguaje de Programación IV.

Ingeniería en Desarrollo de Software

academiaglobal

TUTOR: Aarón Iván Salazar Macías.

ALUMNO: Manuel Marin Sanchez.

FECHA: 31/03/2026.

Indicé.

Portada ……. 1
Indicé …… 2
Introducción …….. 3
Investigación …….. 4,5,6,7
Desarrollo …… 8,9.10
Conclusiones ……. 11
Referencias …… 12

Introducción.

La presente actividad corresponde a la segunda parte del desarrollo del sistema bancario para Banco Mexicano, continuando con el trabajo iniciado en la Actividad 2. En esta etapa, se completa la funcionalidad del programa implementando las opciones de consulta de saldo y cierre de sesión, las cuales se suman a las operaciones de depósito y retiro ya desarrolladas anteriormente.
El programa mantiene los principios de la Programación Orientada a Objetos, con una clase que encapsula los datos financieros y proporciona métodos seguros para su manipulación. La clase principal BancoMexicanoApp gestiona la interfaz con el usuario a través de un menú interactivo por consola, implementando un ciclo que mantiene el programa en ejecución hasta que el usuario selecciona la opción de salir.
La consulta de saldo permite al cliente visualizar su saldo actual de manera clara y sencilla, mientras que la opción de salir muestra un mensaje de despedida y finaliza la ejecución del programa. Ambas funcionalidades se integran de manera coherente con las ya existentes, ofreciendo una experiencia completa al usuario.

Investigación.
Cómo declarar una clase abstracta y heredar sus propiedades y métodos a subclases en los siguientes lenguajes de programación:

1. Java:

```
// Clase abstracta
public abstract class Figura {
    protected String color;

    public Figura(String color) {
        this.color = color;
    }

    // Método abstracto (sin implementación)
    public abstract double calcularArea();

    // Método concreto (con implementación)
    public String getColor() {
        return color;
    }
}

// Subclase que hereda
public class Circulo extends Figura {
    private double radio;

    public Circulo(String color, double radio) {
        super(color);
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```

2. Python:

```
from abc import ABC, abstractmethod

# Clase abstracta
class Figura(ABC):
    def __init__(self, color):
        self.color = color

    # Método abstracto
    @abstractmethod
    def calcular_area(self):
        pass

    # Método concreto
    def get_color(self):
```

```
        return self.color

# Subclase que hereda
class Circulo(Figura):
    def __init__(self, color, radio):
        super().__init__(color)
        self.radio = radio

    def calcular_area(self):
        return 3.1416 * self.radio * self.radio
```

3. C#:

```
// Clase abstracta
public abstract class Figura
{
    protected string color;

    public Figura(string color)
    {
        this.color = color;
    }

    // Método abstracto
    public abstract double CalcularArea();

    // Método concreto
    public string GetColor()
    {
        return color;
    }
}

// Subclase que hereda
public class Circulo : Figura
{
    private double radio;

    public Circulo(string color, double radio) : base(color)
    {
        this.radio = radio;
    }

    public override double CalcularArea()
    {
        return Math.PI * radio * radio;
    }
}
```

4. C++:

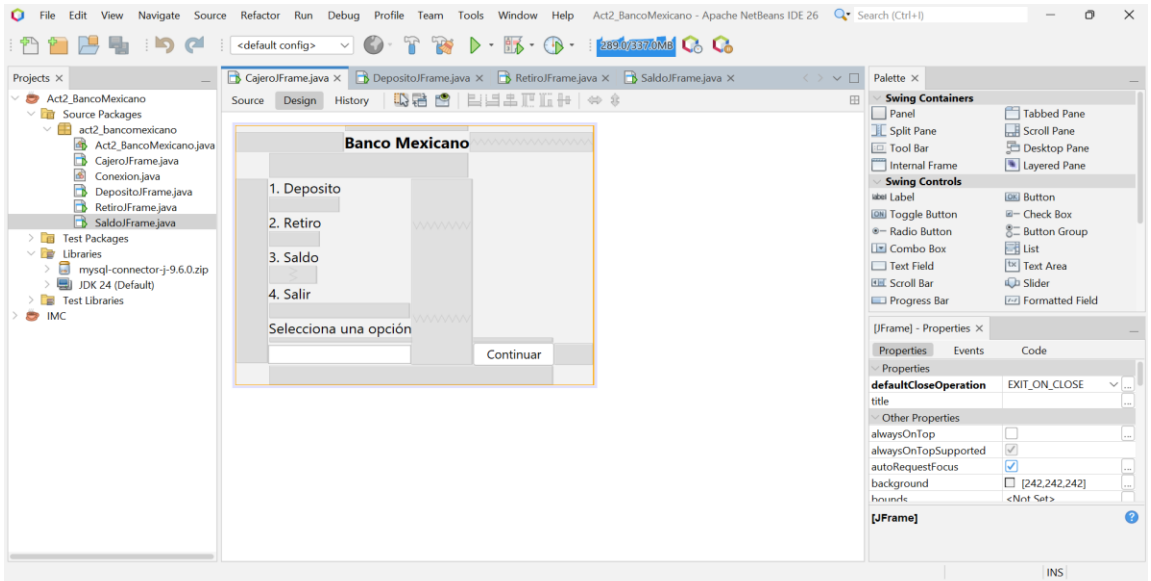
```
#include <iostream>
#include <cmath>
using namespace std;

// Clase abstracta
class Figura {
protected:
    string color;

public:
    Figura(string color) : color(color) {}

    // Método virtual puro (abstracto)
    virtual double calcularArea() = 0;

    // Método concreto
    string getColor() { return color; }
};

// Subclase que hereda
class Circulo : public Figura {
private:
    double radio;

public:
    Circulo(string color, double radio) : Figura(color), radio(radio) {}

    double calcularArea() override {
        return M_PI * radio * radio;
    }
};
```

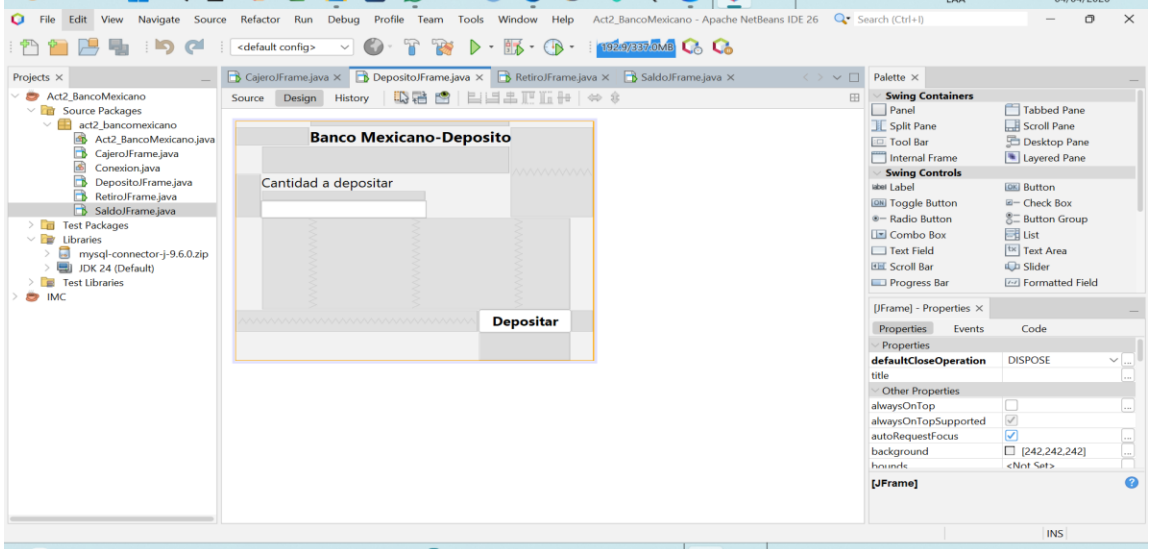
5. PHP:

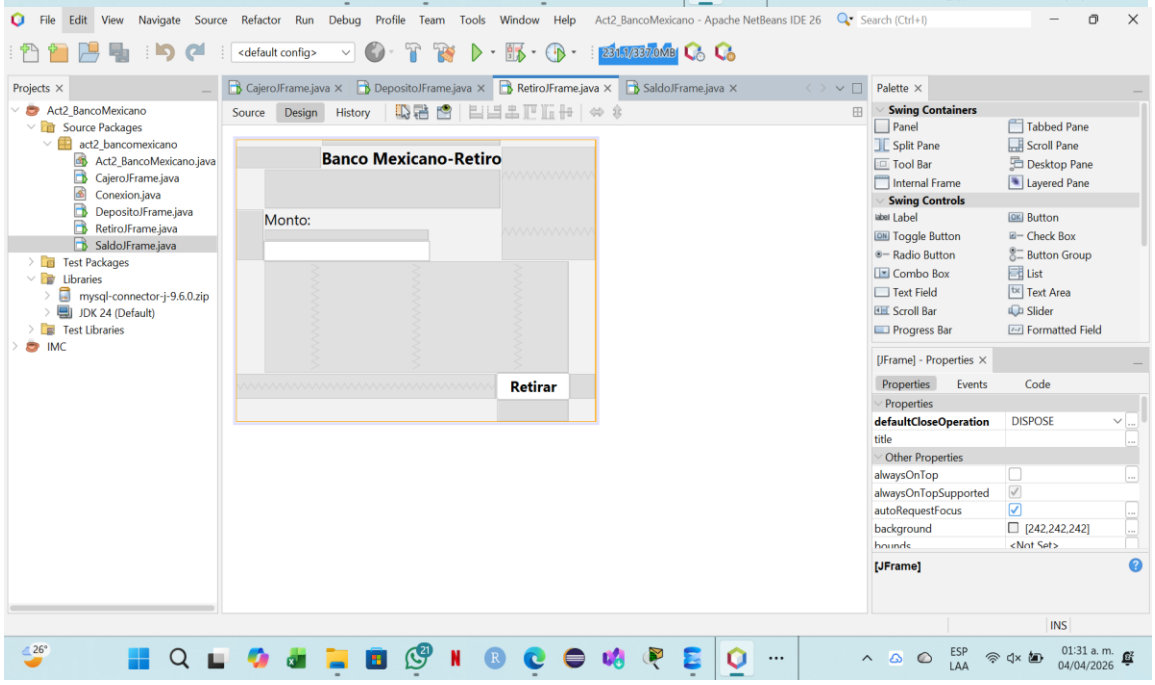
```
<?php
// Clase abstracta
abstract class Figura {
    protected $color;

    public function __construct($color) {
        $this->color = $color;
    }

    // Método abstracto
    abstract public function calcularArea();

    // Método concreto
    public function getColor() {
        return $this->color;
    }
}
```

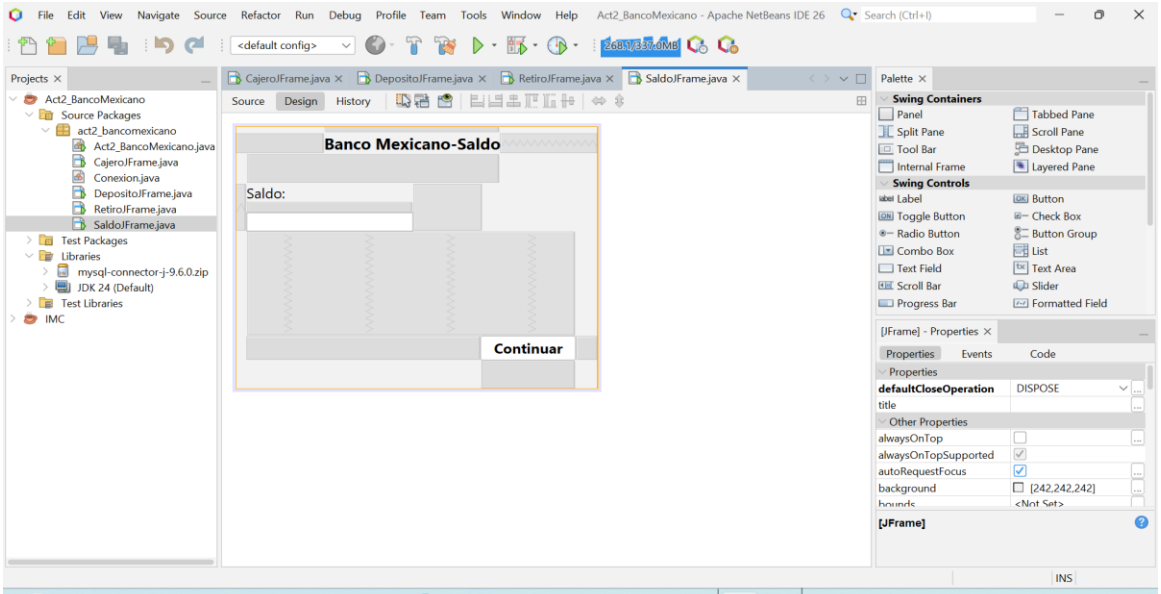
```php
// Subclase que hereda
class Circulo extends Figura {
    private $radio;

    public function __construct($color, $radio) {
        parent::__construct($color);
        $this->radio = $radio;
    }

    public function calcularArea() {
        return M_PI * $this->radio * $this->radio;
    }
}
?>
```

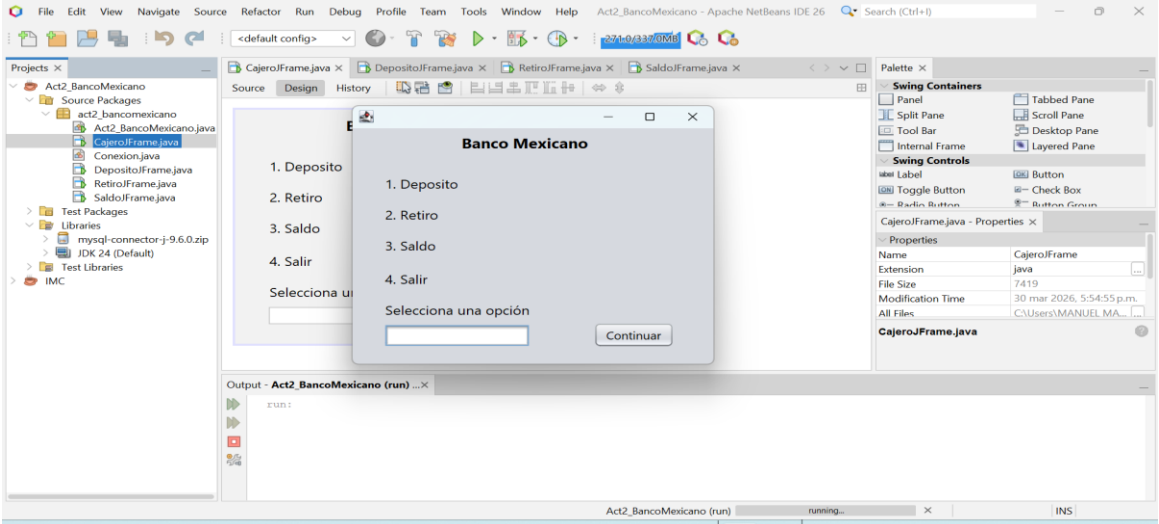
Desarrollo.

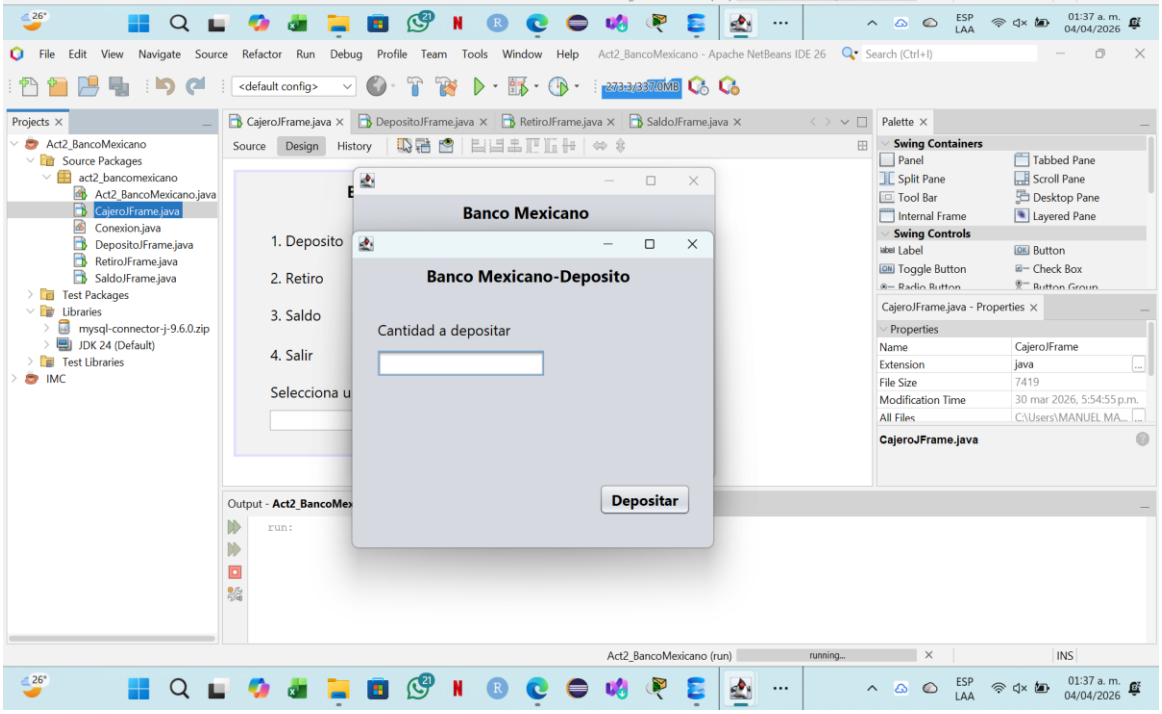

Banco Mexicano
1. Deposito
2. Retiro
3. Saldo
4. Salir
Selecciona una opción
Continuar

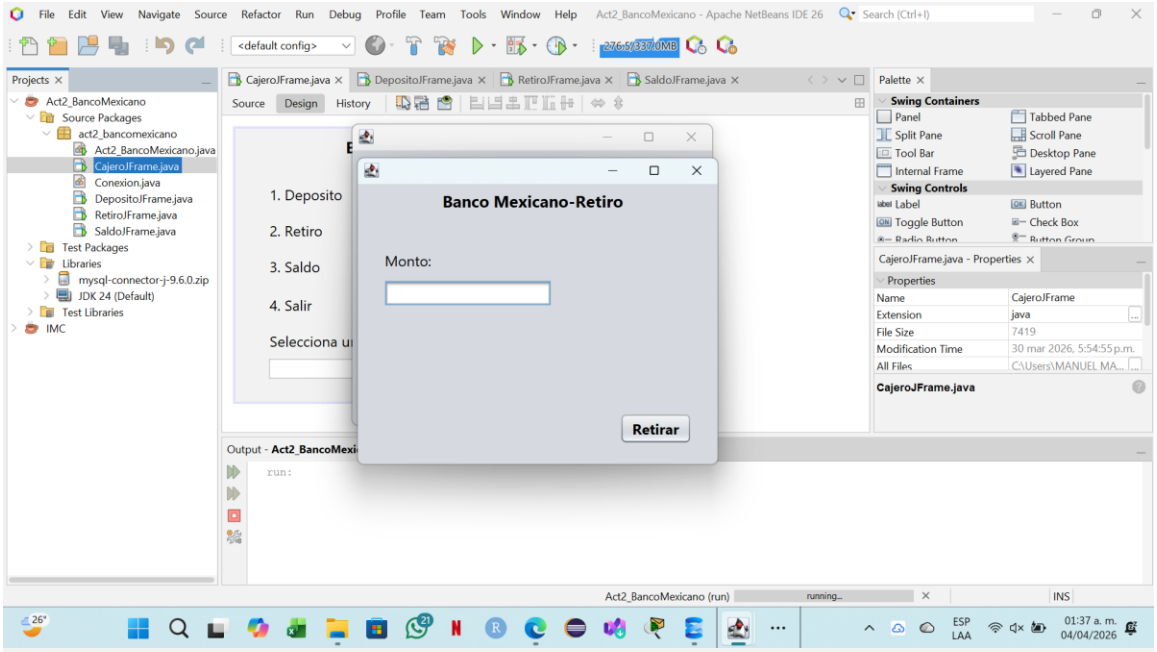

Banco Mexicano-Deposito
Cantidad a depositar
Depositar

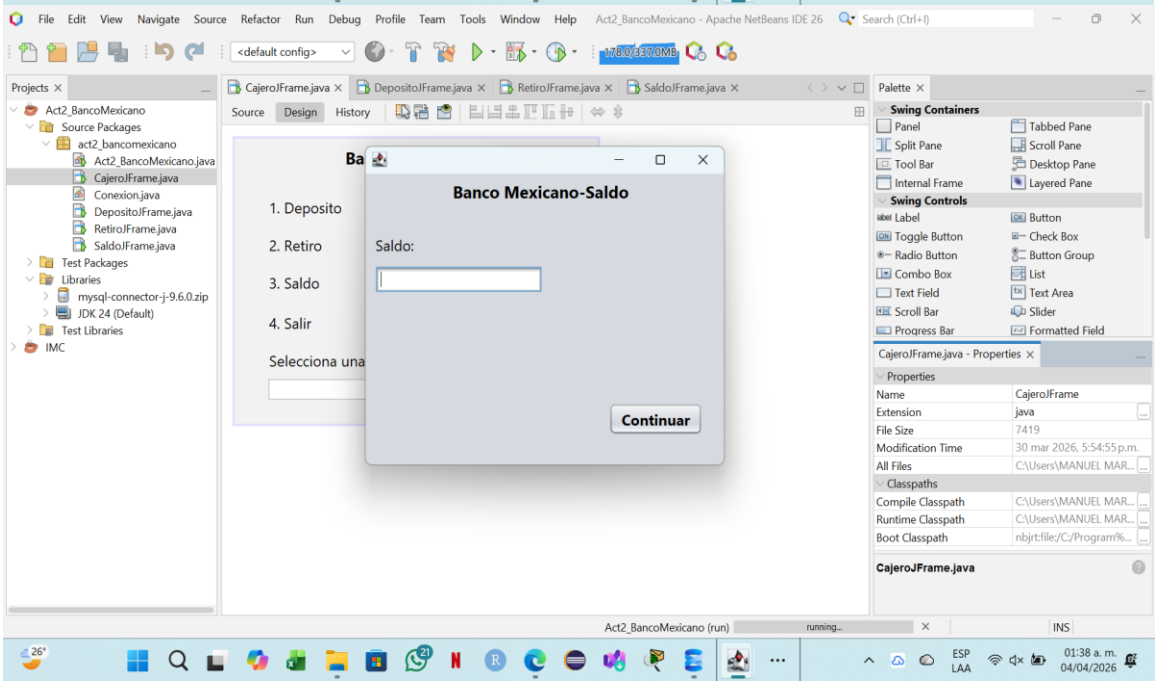

Banco Mexicano-Retiro
Monto:
Retirar


Banco Mexicano-Saldo
Saldo:
Continuar


Banco Mexicano
1. Deposito
2. Retiro
3. Saldo
4. Salir
Selecciona una opción
Continuar


Banco Mexicano
1. Deposito
2. Retiro
3. Saldo
4. Salir
Banco Mexicano-Deposito
Cantidad a depositar
Depositar


Act2_BancoMexicano - Apache NetBeans IDE 26
Banco Mexicano-Retiro
Monto:
Retirar
1. Deposito
2. Retiro
3. Saldo
4. Salir
Selecciona u
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
Act2_BancoMexicano
Source Packages
act2_bancomexicano
Act2_BancoMexicano.java
Conexion.java
Test Packages
Libraries
mysql-connector-j-9.6.0.zip
JDK 24 (Default)
Test Libraries
IMC
Output - Act2_BancoMexi
run:
Act2_BancoMexicano (run)
running...
01:37 a. m.
04/04/2026


Act2_BancoMexicano - Apache NetBeans IDE 26
Banco Mexicano-Saldo
Saldo:
Continuar
1. Deposito
2. Retiro
3. Saldo
4. Salir
Selecciona una
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
Act2_BancoMexicano
Source Packages
act2_bancomexicano
Act2_BancoMexicano.java
Conexion.java
Test Packages
Libraries
mysql-connector-j-9.6.0.zip
JDK 24 (Default)
Test Libraries
IMC
Act2_BancoMexicano (run)
running...
01:38 a. m.
04/04/2026

Conclusiones.

El desarrollo de esta segunda parte del sistema bancario permitió completar el conjunto de funcionalidades requeridas por Banco Mexicano, integrando de manera exitosa las operaciones de depósito, retiro, consulta de saldo y cierre de sesión en una aplicación cohesiva y funcional.

Uno de los aprendizajes más importantes fue la consolidación del uso de estructuras de control como do-while para el menú principal y while para permitir múltiples depósitos consecutivos. La implementación de la validación de saldo en los retiros demostró la importancia de proteger al usuario de operaciones no autorizadas, mientras que la consulta de saldo mostró cómo presentar información de manera clara y accesible.

La aplicación del paradigma de Programación Orientada a Objetos se vio reflejada en la correcta separación de responsabilidades: la clase CuentaBancaria se encarga de la lógica de negocio (depósitos, retiros, consulta de saldo) mientras que la clase BancoMexicanoApp gestiona la interacción con el usuario. Esta separación facilita el mantenimiento y la escalabilidad del sistema.

En conclusión, esta actividad permitió aplicar de manera práctica los conceptos de encapsulamiento, métodos, constructores y estructuras de control en un contexto real, sentando las bases para futuras mejoras como la persistencia de datos o la implementación de una interfaz gráfica.

Referencias.

1. Cosmina, L. (2018). *Java for Absolute Beginners: learn to program the fundamentals the Java 9+ ways*. Apress.
2. Deitel, P. J., & Deitel, H. M. (2016). *Java How to Program, Early Objects* (10th ed.). Pearson.
3. Oracle Corporation. (2023). *The Java™ Tutorials*. https://docs.oracle.com/javase/tutorial/
4. Schildt, H. (2018). *Java: The Complete Reference* (11th ed.). McGraw-Hill Education.
5. Sierra, K., & Bates, B. (2020). *OCA/OCP Java SE 8 Programmer Certification Kit*. John Wiley & Sons.

https://github.com/ManuelMarin14/Manuel_Marin_Act2.git